Thiết kế bộ điều khiển mờ cho robot di động bám tường sử dụng Arduino Uno R3

GVHD: PGS.TS. Huỳnh Thái Hoàng

Ngày 12 tháng 12 năm 2025

Sinh viên thực hiện - Nhóm 14

STT	Họ tên sinh viên	MSSV
1	Võ Đức Bảo	2210288
2	Lê Văn Nhật	2212393
3	Trương Đình Siêu	2212926
4	Đinh Cao Thiên	2213227

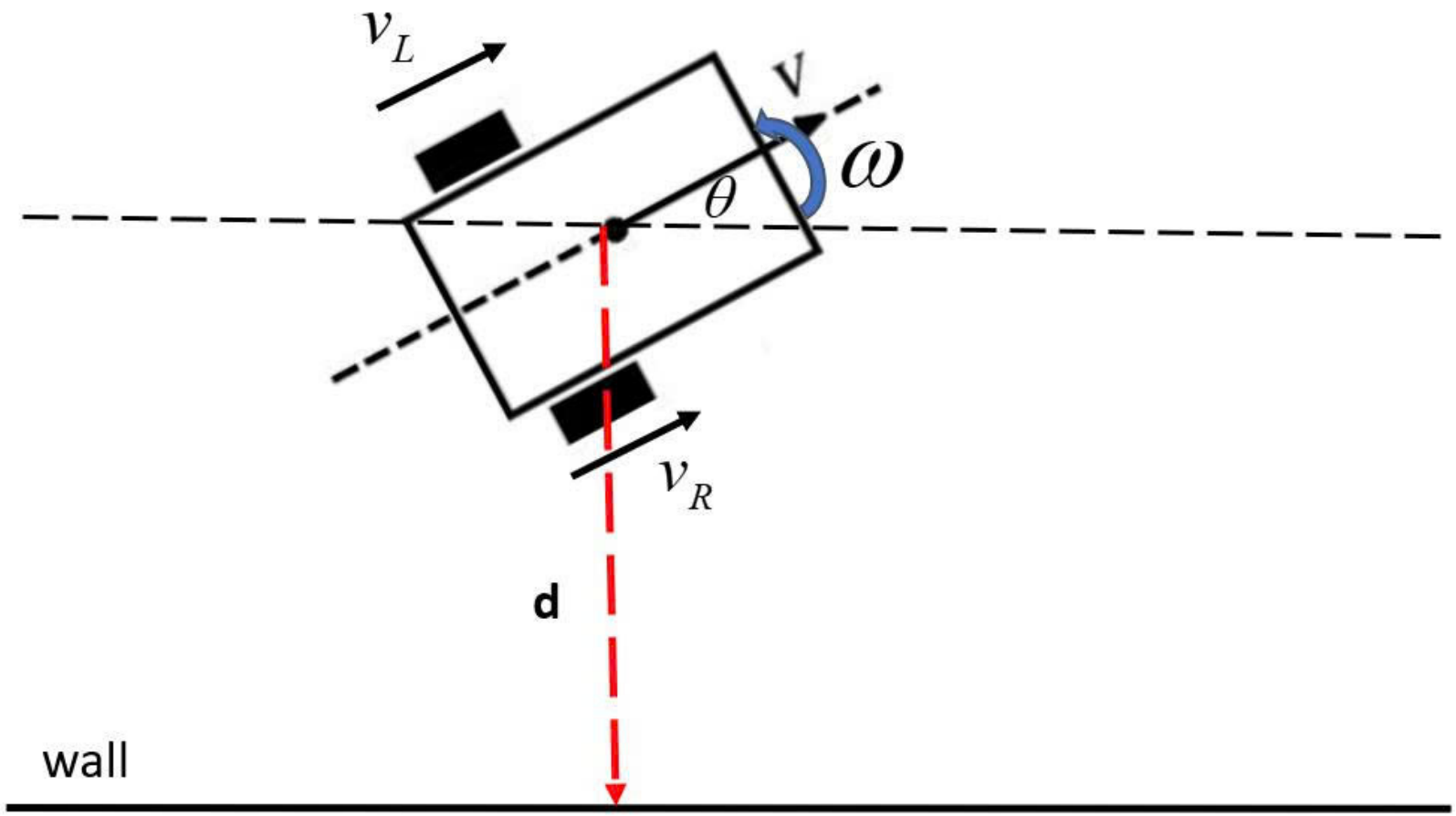
Mục lục

1. Mục tiêu
2. Mô hình động học
3. Phương pháp đề xuất
4. Thiết kế hệ thống điều khiển
5. Thực nghiệm
6. Kết luận

Mục tiêu

- Thiết kế bộ điều khiển mờ điều khiển robot di động giữ khoảng cách cố định so với tường.
- Robot sử dụng cơ cấu lái vi sai, điều khiển tốc độ hai bánh trái/phải.

Mô hình động học của robot vi sai



Mô hình động học của robot vi sai

1. Vận tốc dài và vận tốc góc

$$v = \frac{v_R + v_L}{2}, \qquad \omega = \frac{v_R - v_L}{L}$$

2. Động học vị trí

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega$$

3. Quan hệ vận tốc bánh xe

$$v_R = \omega_R \cdot r, \qquad v_L = \omega_L \cdot r$$

Mô hình động học của robot vi sai

4. Mô hình tổng quát

$$\begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} = \begin{pmatrix} \frac{r}{2}\cos\theta & \frac{r}{2}\cos\theta \\ \frac{r}{2}\sin\theta & \frac{r}{2}\sin\theta \\ \frac{r}{L} & -\frac{r}{L} \end{pmatrix} \begin{pmatrix} \omega_R \\ \omega_L \end{pmatrix}$$

Trong đó:

- v : vận tốc dài của robot.
- ω : vận tốc góc của robot.
- v_R, v_L : vận tốc dài của bánh phải và bánh trái.
- ω_R, ω_L : vận tốc góc của bánh phải và bánh trái.
- r : bán kính bánh xe.
- L : khoảng cách giữa hai bánh.
- x, y : tọa độ vị trí của robot trong hệ trục mặt phẳng.
- θ: góc định hướng của robot.

Phương pháp đề xuất: Điều khiển vận tốc vi sai bằng Fuzzy

Luật điều khiển vận tốc bánh phải:

$$V_R^{set} = V_{\text{BASE}} + \Delta\omega_R \cdot r$$

Luật điều khiển vận tốc bánh trái:

$$V_L^{set} = V_{\text{BASE}} + \Delta\omega_L \cdot r$$

Trong đó:

- V_{BASE}: vận tốc chuẩn khi đi thẳng.
- $\Delta\omega_R, \Delta\omega_L$: biến thiên vận tốc góc do bộ điều khiển mờ sinh ra.
- r: bán kính bánh xe.

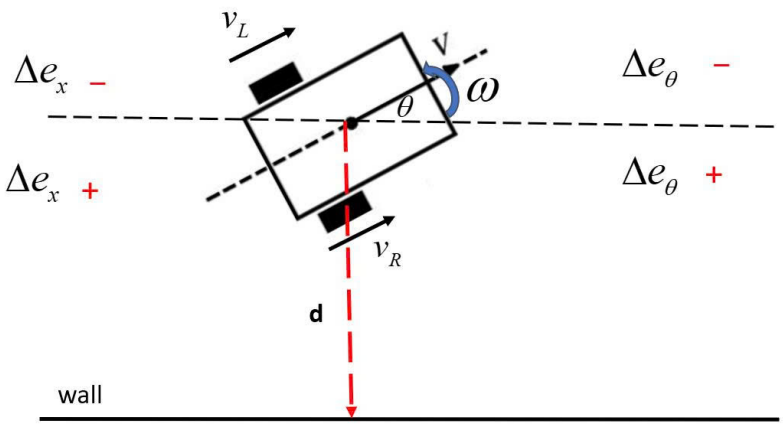
Các biến vào/ra của bộ điều khiển mờ

Biến vào:

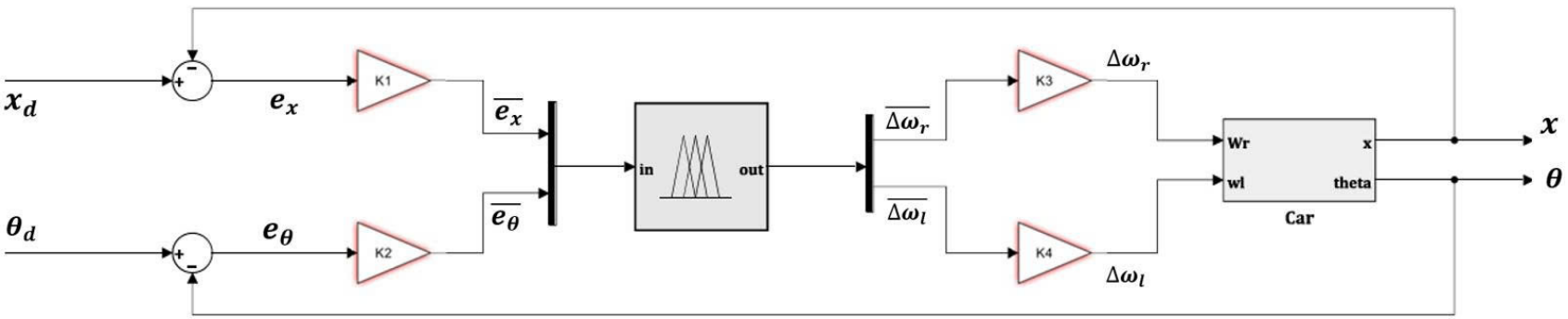
- Sai số khoảng cách e_x
- Sai số góc lệch e_θ

Biến ra:

- $\Delta\omega_L$: biến thiên vận tốc góc bánh trái
- $\Delta\omega_R$: biến thiên vận tốc góc bánh phải



Sơ đồ khối hệ thống điều khiển



Chuẩn hóa các biến vào/ra

1. Sai số khoảng cách

$$0.2 \leq x \leq 2.0 \quad (m)$$

$$\rightarrow K_1 = \frac{1}{1.8}$$

2. Sai số góc

$$-\frac{\pi}{6} \leq \theta \leq \frac{\pi}{6}$$

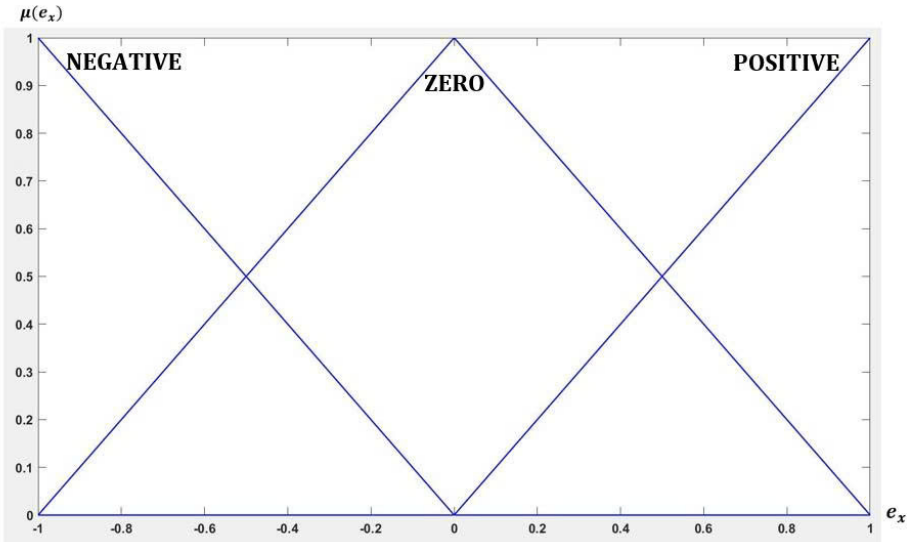
$$\rightarrow K_2 = \frac{6}{\pi}$$

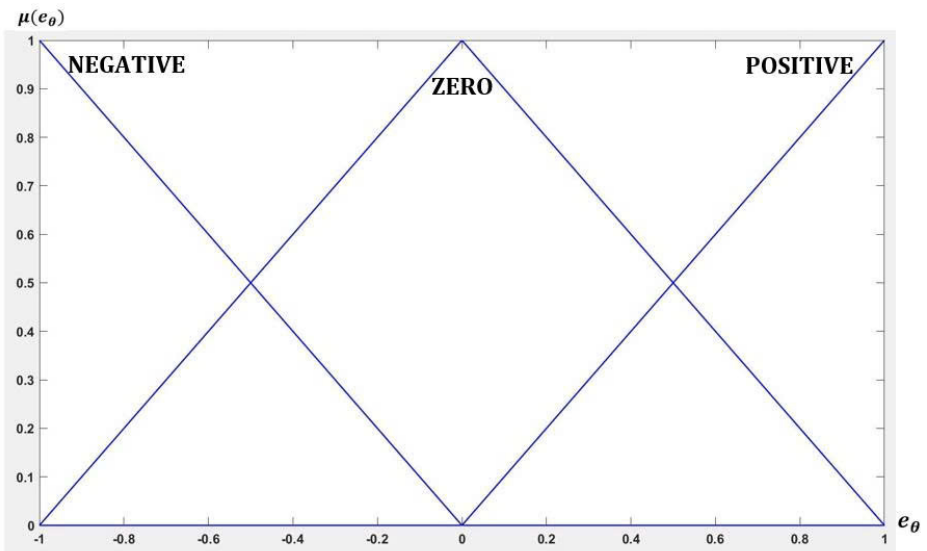
3. Biến thiên vận tốc góc điều khiển

$$-3.5 \leq \Delta\omega_L,\ \Delta\omega_R \leq 3.5 \quad (rad/s)$$

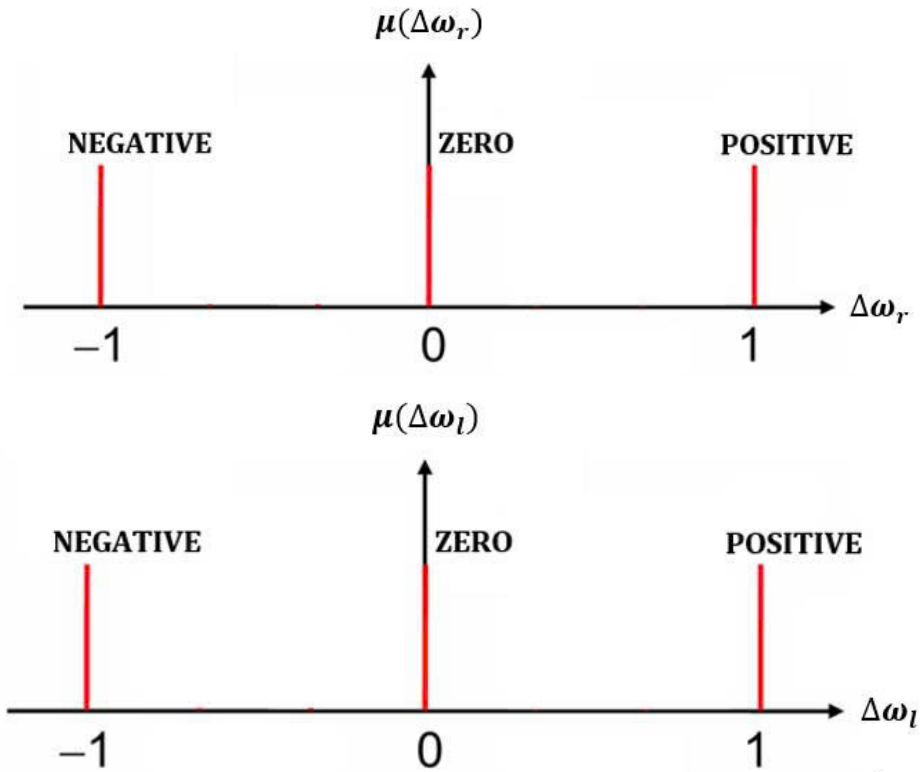
$$\rightarrow K_3, K_4 = 3.5$$

Định nghĩa các giá trị ngôn ngữ

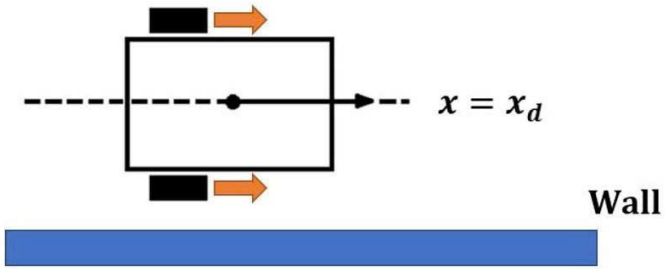




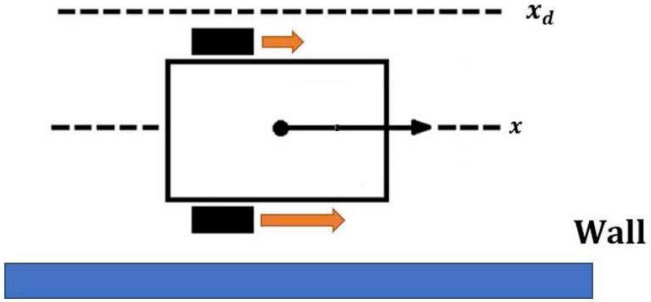
Định nghĩa các giá trị ngôn ngữ



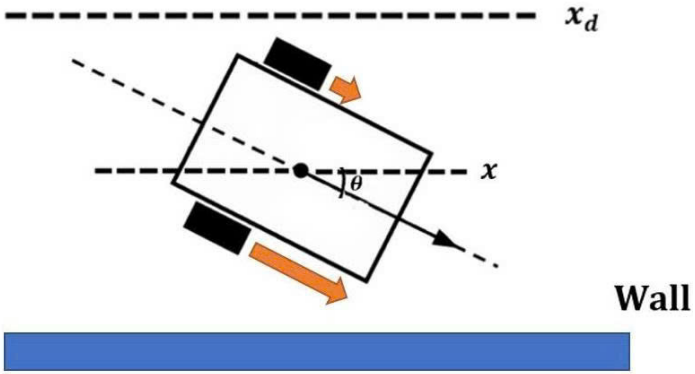
Quy tắc điều khiển dựa trên kinh nghiệm



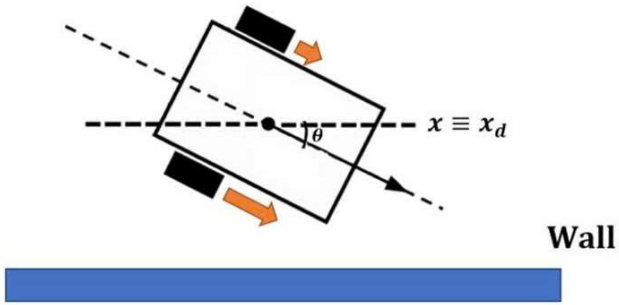
$$\begin{cases} e_\theta = ZE \\ e_x = ZE \end{cases} \rightarrow \begin{cases} \Delta\omega_r = Zero \\ \Delta\omega_l = Zero \end{cases}$$



$$\begin{cases} e_\theta = ZE \\ e_x = POSITIVE \end{cases} \rightarrow \begin{cases} \Delta\omega_r = Positive \\ \Delta\omega_l = Zero \end{cases}$$



$$\begin{cases} e_\theta = POSITIVE \\ e_x = POSITIVE \end{cases} \rightarrow \begin{cases} \Delta\omega_r = Positive \\ \Delta\omega_l = Negative \end{cases}$$



$$\begin{cases} e_\theta = POSITIVE \\ e_x = ZE \end{cases} \rightarrow \begin{cases} \Delta\omega_r = Zero \\ \Delta\omega_l = Negative \end{cases}$$

Bảng quy tắc điều khiển

$\Delta\omega_r$		e_θ		
		Negative	Zero	Positive
e_x	Negative	Negative	Zero	Positive
	Zero	Negative	Zero	Positive
	Positive	Zero	Zero	Negative

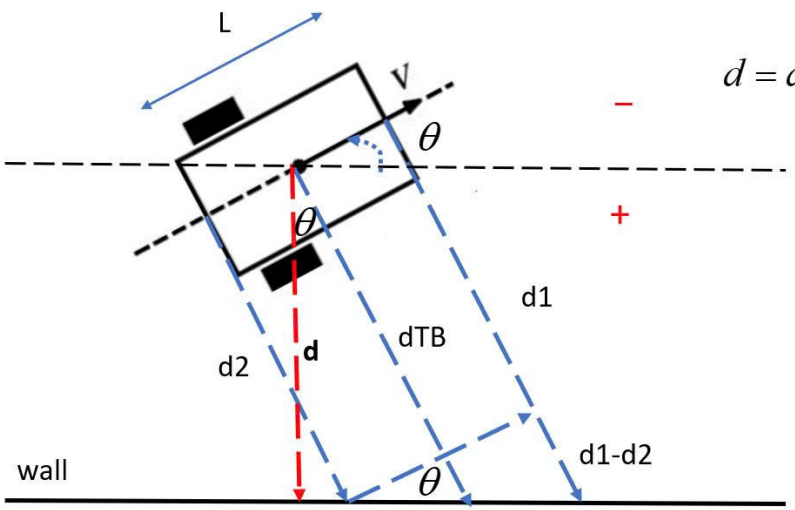
$\Delta\omega_l$		e_θ		
		Negative	Zero	Positive
e_x	Negative	Positive	Positive	Zero
	Zero	Zero	Zero	Negative
	Positive	Zero	Zero	Negative

Phương pháp suy luận: MAX-MIN
Phương pháp giải mờ: Trung bình có trọng số.

Phương pháp đo khoảng cách và góc lệch

Xử lý tín hiệu từ cảm biến

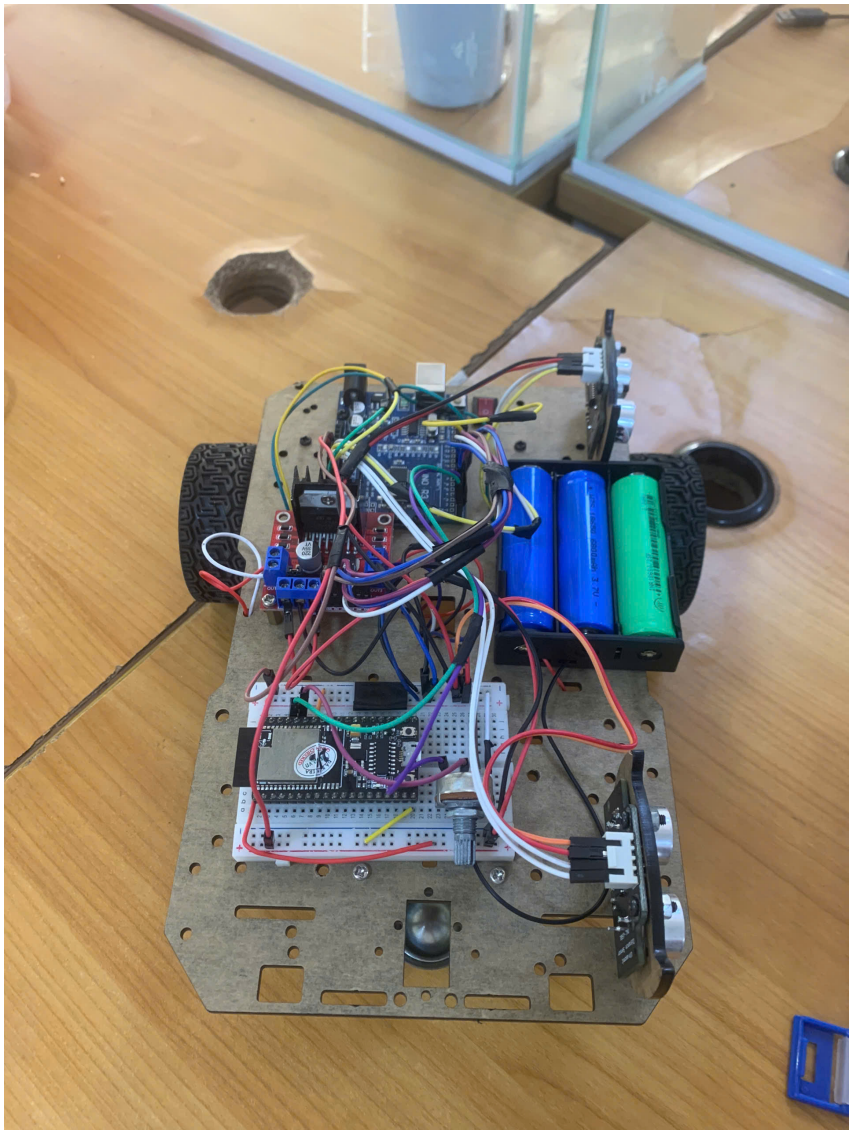
Trên thực tế, việc chỉ sử dụng một cảm biến siêu âm khiến robot không thể đo chính xác khoảng cách vuông góc khi thân xe bị nghiêng. Vì vậy, nhóm đề xuất bố trí hai cảm biến ở phía trước và sau để đồng thời ước lượng được cả góc lệch và khoảng cách vuông góc một cách tin cậy hơn.



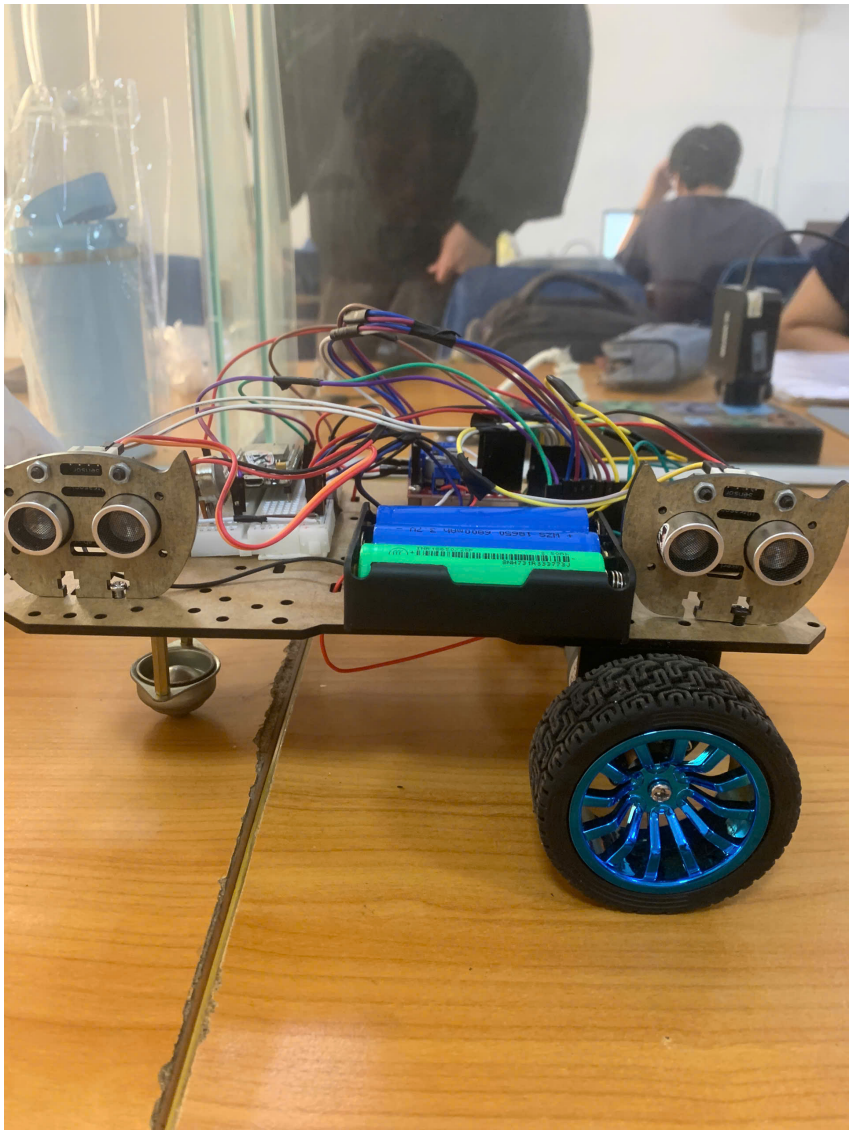
$$\theta = \arctan\left(\frac{d2 - d1}{L}\right)$$

$$d = d_{TB}\cos(\theta) = \frac{d_1 + d_2}{2}\cos(\theta)$$

Mô hình xe thực tế



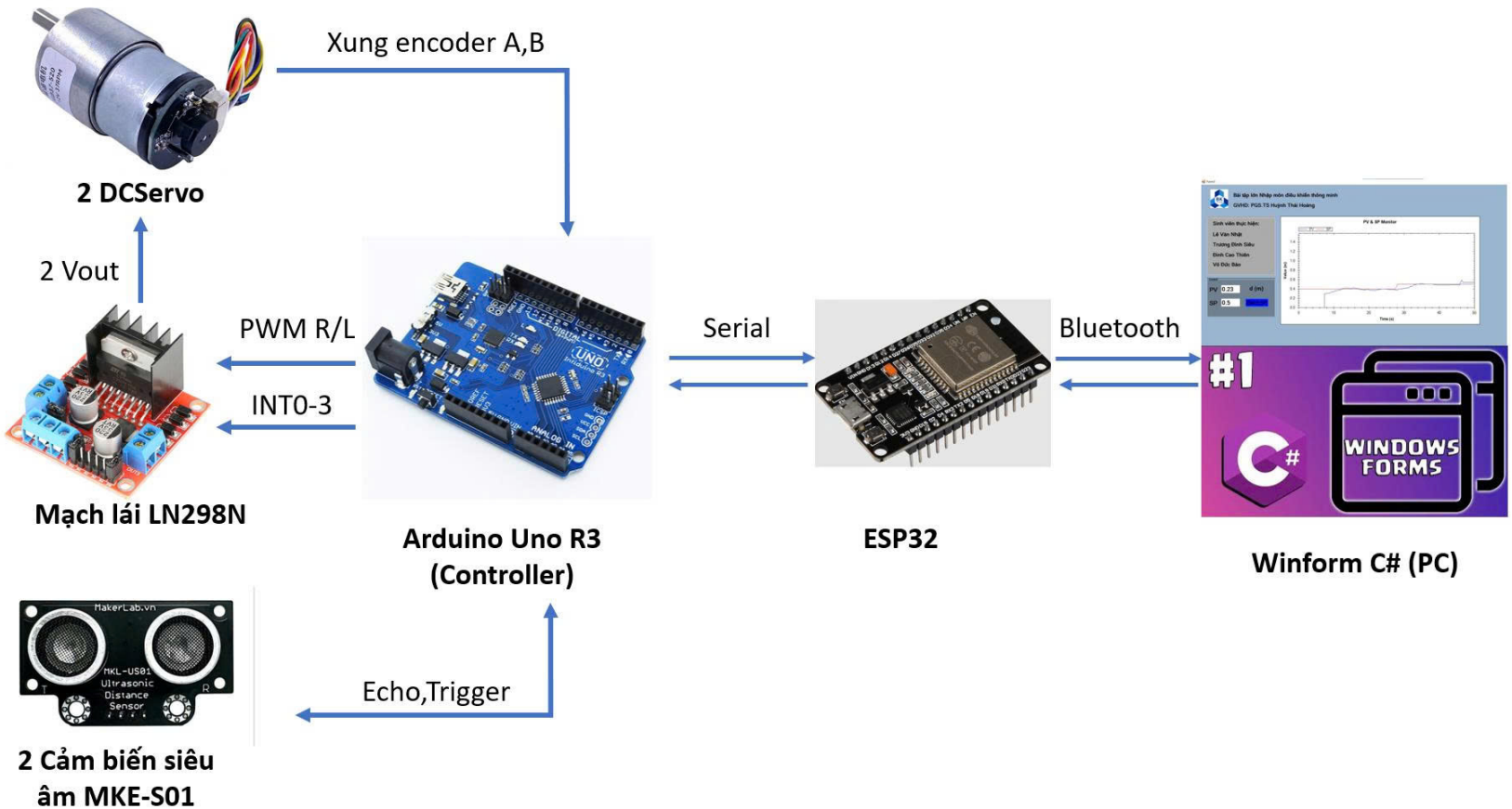
Mô hình xe thực tế



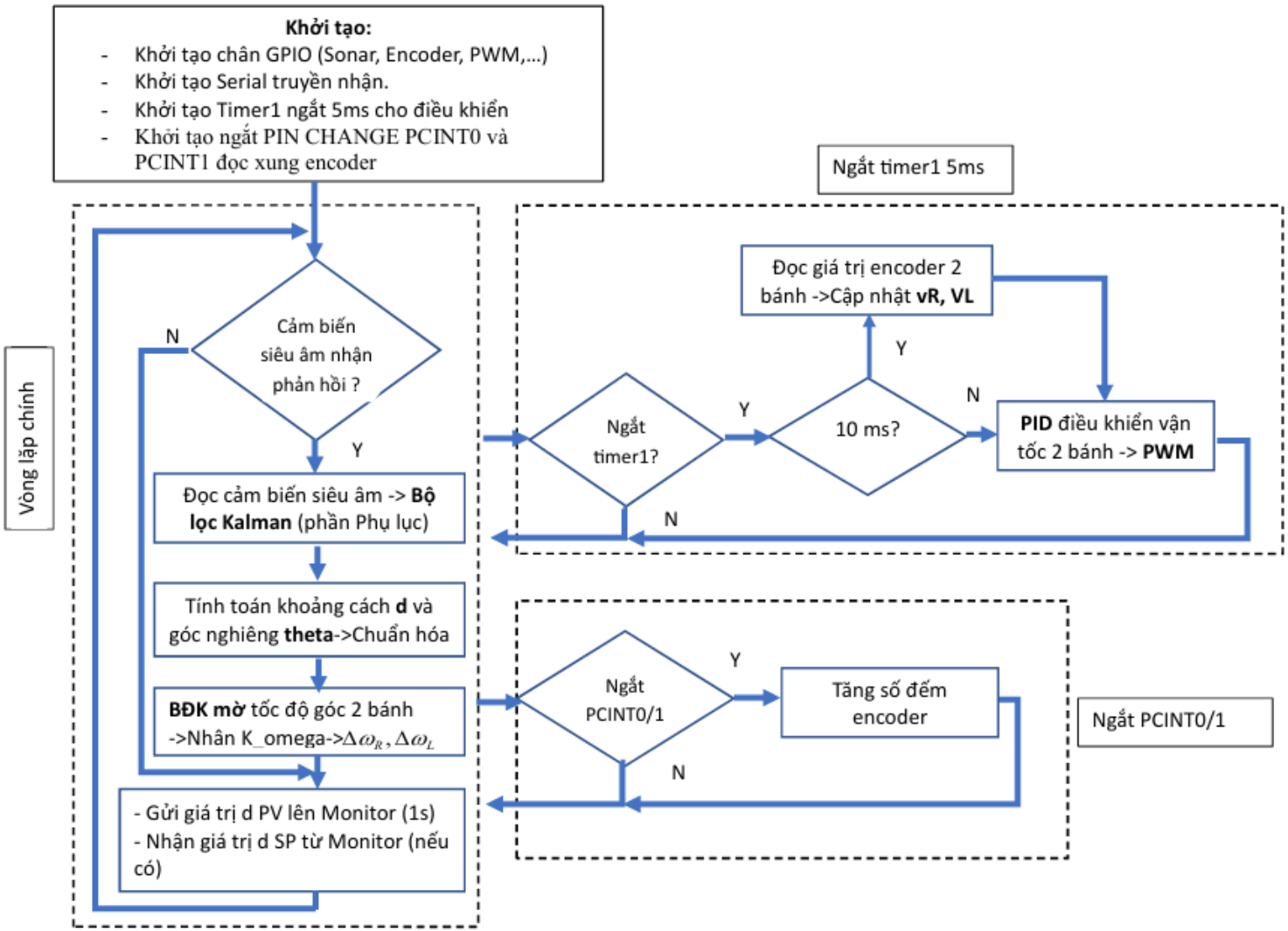
Các thành phần phần cứng và thông số cơ khí

- 2 động cơ DC Servo JGB37-520 12V 333RPM + bánh xe 65mm
- 3 pin Li-ion 3.7V
- 1 mạch lái L298N
- 2 cảm biến siêu âm MKE-S01
- 1 Arduino Uno R3
- 1 ESP32
- Chiều dài giữa 2 bánh xe L = 20cm
- Bán kính bánh xe r = 32.5mm

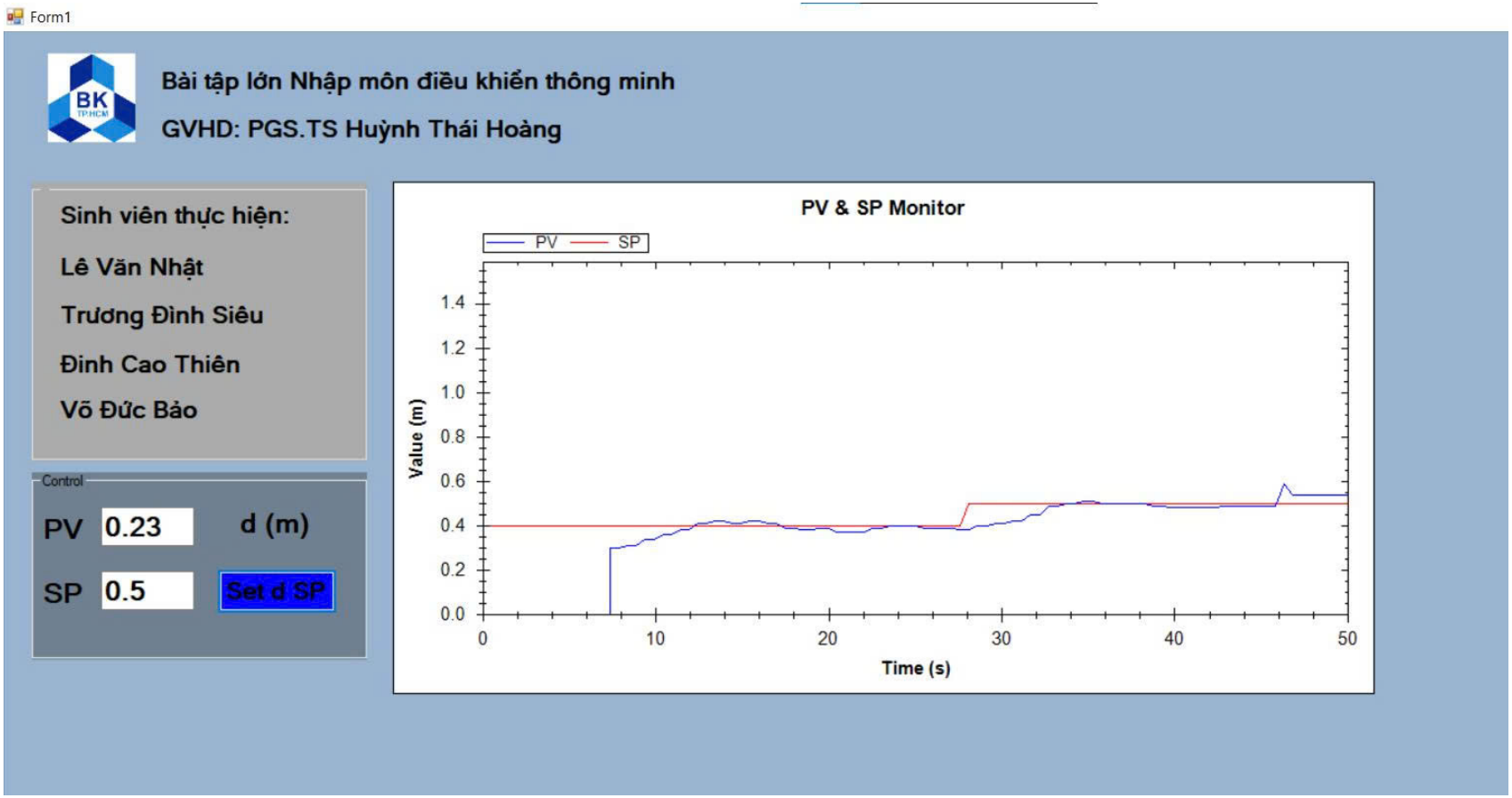
Sơ đồ khối phần cứng



Giải thuật điều khiển



Kết quả thực nghiệm



Kết luận

Kết luận:

- Đề tài đã xây dựng thành công bộ điều khiển mờ điều khiển vận tốc vi sai cho robot, tuy nhiên đáp ứng của hệ thống chưa tốt trong nhiều trường hợp.

Hướng phát triển:

- Nếu tiếp tục, có thể thêm nhiều dữ liệu đầu vào như biến thiên sai số khoảng cách hoặc biến thiên sai số góc đặt để chỉnh định bộ điều khiển hoạt động tốt hơn và viết thêm nhiều luật mờ hơn để tăng độ chính xác.
- Có thể kết hợp bài toán tự động né vật cản trong tương lai.

Tài liệu tham khảo

[1.] PGS. TS. Huỳnh Thái Hoàng, Bài giảng *"NHẬP MÔN ĐIỀU KHIỂN THÔNG MINH"*, Trường Đại học Bách Khoa TP.HCM.

[2.] Vamsi Mohan Peri, Luận văn thạc sĩ *"Fuzzy Control Logic Controller for an Autonomous Mobile Robot"*, Đại học Công nghệ Jawaharlal Nehru, 2002.

Phụ lục

Mã nguồn Kalman Filter cho cảm biến phía trước

```
float Q1 = 0.1;   // nhiễu mô hình
float R1 = 3;     // nhiễu đo
float x1 = 0;     // giá trị ước lượng
float P1 = 1;     // sai số ban đầu

float kalman_update_d1(float z1)
{
    // Prediction
    float x1_pred = x1;
    float P1_pred = P1 + Q1;

    // Kalman gain
    float K1 = P1_pred / (P1_pred + R1);

    // Update
    x1 = x1_pred + K1 * (z1 - x1_pred);
    P1 = (1 - K1) * P1_pred;

    return x1;
}
```

Phụ lục

Mã nguồn Kalman Filter cho cảm biến phía sau

```
float Q2 = 0.1;   // nhiễu mô hình
float R2 = 3;     // nhiễu đo
float x2 = 0;     // giá trị ước lượng
float P2 = 1;     // sai số ban đầu

float kalman_update_d2(float z2)
{
    // Prediction
    float x2_pred = x2;
    float P2_pred = P2 + Q2;

    // Kalman gain
    float K2 = P2_pred / (P2_pred + R2);

    // Update
    x2 = x2_pred + K2 * (z2 - x2_pred);
    P2 = (1 - K2) * P2_pred;

    return x2;
}
```